

MessageCache

C# Developer. Advanced



Защита проекта

Тема: MessageCache



Лев Аксёнов

.NET разработчик в области Desktop-приложений

Цель и задачи проекта

Цель проекта: создать высокопроизводительный In-Memory Key-Value cache с возможностью подписки клиентов на изменение значения в кэше

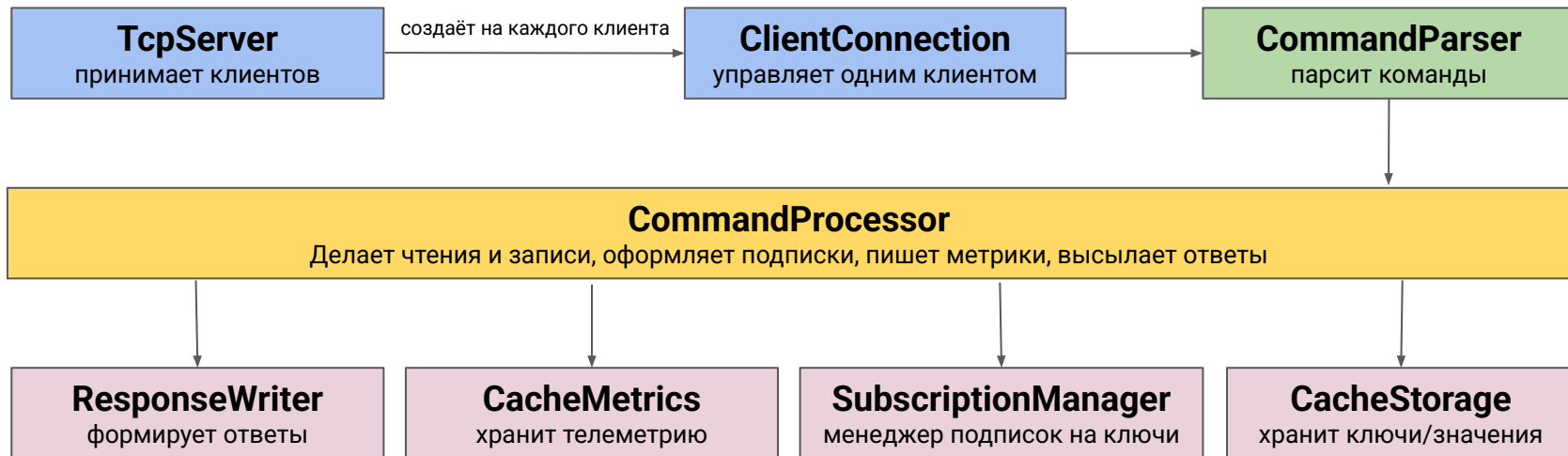
Задачи проекта:

1. Реализовать ядро хранилища данных.
2. Реализовать zero-allocation парсер команд.
3. Реализовать сетевой уровень общения сервера и клиента.
4. Реализовать обработчик команд от клиентов.
5. Добавить телеметрию.
6. Создать бенчмарки и нагрузочные тесты.

Используемые технологии

Название	Назначение
.NET 8	Среда исполнения сервиса
BenchmarkDotNet	Измерение производительности сервиса
OpenTelemetry	Просмотр данных о работе сервиса
NBomber	Нагрузочное тестирование сервиса

Архитектура сервиса



Бенчмарки

Method	KeyCount	ThreadCount	Mean	Error	StdDev	Median	Min	Max	Ratio	RatioSD	Gen0	Completed Work Items	Lock Contentions	Allocated	Alloc Ratio
SingleThreadGet	1000	1	35.09 ns	0.585 ns	0.547 ns	35.20 ns	33.92 ns	35.80 ns	1.00	0.02	-	0.0000	-	-	NA
SingleThreadSet	1000	1	26.23 ns	0.468 ns	0.415 ns	26.11 ns	25.80 ns	26.95 ns	0.75	0.02	-	0.0000	-	-	NA
SingleThreadDelete	1000	1	66.67 ns	0.216 ns	0.169 ns	66.71 ns	66.37 ns	66.91 ns	1.90	0.03	-	-	-	-	NA
SetWithExpiry	1000	1	52.39 ns	0.837 ns	0.782 ns	51.99 ns	51.80 ns	54.45 ns	1.49	0.03	-	0.0000	-	-	NA
GetAllKeys	1000	1	13,834.62 ns	143.170 ns	133.921 ns	13,843.79 ns	13,612.26 ns	14,080.39 ns	394.37	7.07	1.0071	0.0000	-	16904 B	NA
ConcurrentReads	1000	1	39,673.87 ns	503.214 ns	470.706 ns	39,706.09 ns	38,875.35 ns	40,406.18 ns	1,130.94	21.61	0.0610	1.9960	0.0006	1760 B	NA
ConcurrentMixedReadWrite	1000	1	3,404.08 ns	65.872 ns	102.555 ns	3,411.78 ns	3,228.64 ns	3,618.52 ns	97.04	3.24	0.1030	2.0249	0.0000	1732 B	NA
SingleThreadGet	1000	4	34.18 ns	0.724 ns	1.589 ns	34.12 ns	30.76 ns	37.63 ns	1.00	0.07	-	0.0000	-	-	NA
SingleThreadSet	1000	4	24.81 ns	0.380 ns	0.356 ns	24.94 ns	24.25 ns	25.35 ns	0.73	0.04	-	-	-	-	NA
SingleThreadDelete	1000	4	66.76 ns	1.271 ns	2.625 ns	66.04 ns	64.21 ns	72.93 ns	1.96	0.12	-	-	-	-	NA
SetWithExpiry	1000	4	50.90 ns	1.009 ns	1.036 ns	50.30 ns	49.98 ns	53.12 ns	1.49	0.07	-	0.0000	-	-	NA
GetAllKeys	1000	4	13,618.77 ns	101.273 ns	84.567 ns	13,596.56 ns	13,488.80 ns	13,826.55 ns	399.27	18.55	1.0071	0.0000	-	16904 B	NA
ConcurrentReads	1000	4	38,190.30 ns	757.267 ns	872.069 ns	38,059.33 ns	36,943.27 ns	40,265.33 ns	1,119.66	57.30	0.1221	4.9639	0.0016	2415 B	NA
ConcurrentMixedReadWrite	1000	4	26,611.31 ns	527.725 ns	1,371.628 ns	27,394.48 ns	23,961.67 ns	28,575.97 ns	780.19	53.77	0.1221	4.8419	0.0021	2360 B	NA
SingleThreadGet	1000	8	35.05 ns	0.707 ns	0.756 ns	34.92 ns	33.74 ns	36.26 ns	1.00	0.03	-	0.0000	-	-	NA
SingleThreadSet	1000	8	25.38 ns	0.241 ns	0.214 ns	25.46 ns	24.96 ns	25.67 ns	0.72	0.02	-	0.0000	-	-	NA
SingleThreadDelete	1000	8	66.79 ns	0.897 ns	0.795 ns	66.80 ns	65.61 ns	68.01 ns	1.91	0.05	-	0.0000	-	-	NA
SetWithExpiry	1000	8	53.05 ns	0.514 ns	0.481 ns	53.09 ns	52.26 ns	53.76 ns	1.51	0.03	-	0.0000	-	-	NA
GetAllKeys	1000	8	14,062.07 ns	176.318 ns	137.657 ns	14,082.38 ns	13,707.37 ns	14,260.25 ns	401.39	9.23	1.0071	0.0000	-	16904 B	NA
ConcurrentReads	1000	8	57,750.80 ns	120.488 ns	106.809 ns	57,738.68 ns	57,609.69 ns	57,940.40 ns	1,648.46	34.72	0.1831	8.8799	0.0023	3273 B	NA
ConcurrentMixedReadWrite	1000	8	53,829.70 ns	1,048.932 ns	1,122.346 ns	53,944.21 ns	50,313.89 ns	55,004.76 ns	1,536.54	44.85	-	7.9031	0.0017	3090 B	NA



Выводы

В результате работы над проектом удалось создать высокопроизводительный In-Memory Key-Value кэш с собственным протоколом взаимодействия и возможностью подписки на изменения значений в ключах.

В процессе разработки было применено множество знаний из курса, таких как использование телеметрии, ReaderWriterLockSlim, zero-allocation через ReadOnlySpan, ArrayPool, Benchmarks.



Вопросы и рекомендации



если есть вопросы



если вопросов нет



Спасибо за внимание!